从泛代数到现代 Code Agent

——大模型、Transformer、MCP 与 Skill 的数学本质与系统工程

刘昊岳

2026 年 10 月 17 日

摘要

当前人工智能领域的工程实践（如 Claude Code 等自主编程智能体、MCP 协议、Skill 动态技能架构）往往被包裹在繁杂的工业界术语中。然而从数学与理论计算机科学的第一性原理审视，这些系统有着极其清晰且优雅的代数与分析结构。本讲义以**多类泛代数（Many-Sorted Universal Algebra）**与**余代数（Coalgebra）**为顶层骨架，自顶向下系统性地剖析现代 AI 智能体的四大支柱：（1）泛代数项求值与闭环控制视角下的 Agent 本质；（2）大语言模型（LLM）的概率测度原理与基于形式文法的受限解码（Constrained Decoding）；（3）Transformer 作为希尔伯特空间上的算子族及其 KV Cache 硬件动力学；（4）MCP（外设原子算子协议）与 Skill（渐进式披露的高阶派生算子）的正交解耦机制。通过打通抽象代数、概率测度论、矩阵分析与系统工程，构建一套不可绕过的硬核知识体系。

目录

1 泛代数（Universal Algebra）与 Code Agent 的形式化本质 **3**

1.1 为什么必须用泛代数理解 Code Agent? 3

1.2 多类泛代数（Many-Sorted Algebra）核心公理体系 3

1.3 自由项代数（Term Algebra）与同态求值定理 4

1.4 对偶视角：余代数（Coalgebra）与闭环随机控制 5

2 大语言模型（LLM）原理与受限解码机制 **5**

2.1 自回归生成的测度论与信息论原理 5

2.2 词元化（Tokenization）：自由半群上的子字压缩 6

2.3 不可绕过的隐性基石：受限解码（Constrained Decoding） 6

3 Transformer 架构：希尔伯特空间上的算子动力学 **7**

3.1 Transformer 与 LLM 的精确关系 7

3.2 残差流（Residual Stream）：高维向量空间的总线动力学 7

3.3 缩放点积注意力与方差稳定化证明 8

3.4 旋转位置编码（RoPE）：正交群上的相对位置内积不变性 9

3.5 SwiGLU 前馈网络：参数化键值关联记忆 9

3.6 核心工程基石：KV Cache 递推机制与硬件瓶颈分析 9

4 MCP（Model Context Protocol）原理：泛代数签名的标准化总线 10
4.1 为什么需要 MCP？从 $M \times N$ 耦合灾难到统一接口代数 10
4.2 Host-Client-Server 三层拓扑架构 11
4.3 三大核心原语的形式化代数定义 11
4.4 传输层与 JSON-RPC 2.0 状态机交互全流程 11

5 Skill（智能体技能）原理：渐进式披露与派生算子抽象 13
5.1 为什么仅靠 MCP 会遭遇系统性瓶颈？ 13
5.2 Skill 与 MCP 的正交性：原子算子 vs. 派生宏算子 13
5.3 Skill 的核心机制：三级渐进式披露（Progressive Disclosure） 14

6 补全拼图：支撑现代 Code Agent 的三大隐性工程基石 14
6.1 隐性基石一：编译/测试反馈作为语法空间上的离散梯度（ReAct 闭环） 15
6.2 隐性基石二：子智能体（Sub-agents）与函数调用栈帧隔离 15
6.3 隐性基石三：上下文压缩（Auto-Compaction）与充分统计量投影 15

1 泛代数（Universal Algebra）与 Code Agent 的形式化本质

当我们观察 Claude Code 等现代 Code Agent 时，表面上看到的是“大语言模型在不断读写文件、执行终端命令、修改代码”。但如果我们剥离自然语言的表象，直击其底层计算结构，就会发现一个极其深刻的事实：**任何具备工具调用能力的 Agent 系统，本质上都是一个多类泛代数（Many-Sorted Universal Algebra）与其对偶余代数（Coalgebra）构成的复合动力系统**。

1.1 为什么必须用泛代数理解 Code Agent？

初学者往往将大语言模型（LLM）与执行环境混为一谈。实际上，LLM 本身是一个纯粹的**符号语法生成器（Syntactic Generator）**，它既不能直接修改磁盘上的一个比特，也不能直接发送一个网络数据包；所有的物理副作用（Side Effects）都发生在宿主机操作系统环境中。

泛代数之所以是理解 Agent 不可绕过的数学工具，是因为它精准地分离了两个世界：

(1) **语法世界（Syntax / Term Algebra）**：由符号、变量和函数名拼装而成的形式化表达式树（即 LLM 生成的 JSON Tool Calls、抽象语法树 AST、思维链序列）。

(2) **语义世界（Semantics / Carrier Structure）**：真实的 Linux 文件系统、进程状态空间、Git 提交图与硬件寄存器。

(3) **解释器桥梁（Homomorphism）**：Agent 宿主程序（如 Claude Code 的 Runtime 与 MCP Server）充当了从语法世界到语义世界的**唯一代数同态映射（Evaluation Homomorphism）**。

1.2 多类泛代数（Many-Sorted Algebra）核心公理体系

经典的单类泛代数（如群、环、域）只在一个集合上定义运算。然而在编程与 Agent 系统中，数据具有严格的类型区分（如文件路径 `FilePath`、正则表达式 `Regex`、文件内容 `String`、退出码 `ExitCode`）。因此，我们需要引入**多类泛代数**。

定义 1.1 (多类代数签名 / Many-Sorted Signature)**.** 一个多类签名是一个二元组 $\Sigma = (S, \Omega)$，其中：

- S 是一个非空有限集，称为**类别集（Sort Set，即类型系统）**。
- Ω 是**算子符号集（Operation Symbols）**，每个算子符号 $\omega \in \Omega$ 都被赋予一个元数签名（Arity）：

$$\omega : s_1 \times s_2 \times \cdots \times s_n \longrightarrow s_0, \quad (s_i \in S,\ n \geq 0) \tag{1}$$

当 $n = 0$ 时，$\omega : \emptyset \to s_0$ 称为类型 s_0 上的**常数符号（Constant Symbol）**。

定义 1.2 (Σ-代数结构 / Σ-Algebra)**.** 给定签名 $\Sigma = (S, \Omega)$，一个 Σ-代数结构 $\mathcal{A}$ 由以下两部分组成：

(1) **载体集族（Carrier Sets）**：$A = \{A_s\}_{s \in S}$，为每个类型 $s \in S$ 指定一个具体的非空集合 A_s。

(2) **解释映射（Interpretation）**：为每个算子符号 $\omega : s_1 \times \cdots \times s_n \to s_0$ 关联一个具体的集合映射（即真实可执行函数）：

$$\omega^{\mathcal{A}} : A_{s_1} \times A_{s_2} \times \cdots \times A_{s_n} \longrightarrow A_{s_0} \tag{2}$$

例 1.1 (Claude Code 的泛代数建模)**.** 在 Claude Code 中，整个工具系统的 JSON Schema 声明直接构成了一个多类代数签名 Σ_{CC}：

- **类别集** S：$\{\texttt{EnvState}, \texttt{Path}, \texttt{Content}, \texttt{Cmd}, \texttt{Obs}\}$。其中 `EnvState` 是整个操作系统与工作区的完整状态空间。
- **算子集** Ω：

$$\texttt{Read} : \texttt{EnvState} \times \texttt{Path} \longrightarrow \texttt{EnvState} \times \texttt{Obs}$$

$$\texttt{Edit} : \texttt{EnvState} \times \texttt{Path} \times \texttt{Content} \times \texttt{Content} \longrightarrow \texttt{EnvState} \times \texttt{Obs}$$

$$\texttt{Bash} : \texttt{EnvState} \times \texttt{Cmd} \longrightarrow \texttt{EnvState} \times \texttt{Obs}$$

注意：这揭示了函数式编程与泛代数中的一个核心技巧——通过将环境状态 `EnvState` 显式作为算子的输入与输出分量（即范畴论中的 State Monad $S \to (A \times S)$），我们将带有物理副作用的指令严格纳入了纯泛代数映射的框架中！

1.3 自由项代数（Term Algebra）与同态求值定理

LLM 在单步推理或规划中，并不会直接触碰载体集 $A_{\texttt{EnvState}}$，它只能用符号去拼装“调用计划”。这种符号拼装构成的空间就是**项代数**。

定义 1.3 (项代数 / Term Algebra $T_\Sigma(X)$)**.** 设 $X = \{X_s\}_{s \in S}$ 为与签名 Σ 不交的变量集族。项代数 $T_\Sigma(X) = \{T_{\Sigma,s}(X)\}_{s \in S}$ 是满足以下归纳规则的最小集合族：

(1) **基础步**：若 $x \in X_s$，则 $x \in T_{\Sigma,s}(X)$；若 $c : \emptyset \to s$ 为常数符号，则 $c \in T_{\Sigma,s}(X)$。

(2) **归纳步**：若 $\omega : s_1 \times \cdots \times s_n \to s_0$ 且 $t_i \in T_{\Sigma,s_i}(X)\,(i = 1, \ldots, n)$，则形式符号串 $\omega(t_1, \ldots, t_n) \in T_{\Sigma,s_0}(X)$。

当 $X = \emptyset$ 时，$T_\Sigma(\emptyset)$ 简记为 T_Σ，称为**基项代数（Ground Term Algebra）**。

项代数的伟大之处在于它具有泛代数中最核心的**泛性质（Universal Property，即初始代数性质 Initiality）**：

定理 1.1 (泛代数的自由扩张与唯一同态定理)**.** *对于任意 Σ-代数 $\mathcal{A}$ 以及任意变量赋值映射（Valuation）$v : X \to A$，存在唯一的代数同态（Homomorphism）$[\![\cdot]\!]_v : T_\Sigma(X) \to \mathcal{A}$ 扩张了 v，即对任意算子 $\omega \in \Omega$ 与项 $t_1, \ldots, t_n$，严格满足：*

$$[\![\omega(t_1, \ldots, t_n)]\!]_v = \omega^{\mathcal{A}}\big([\![t_1]\!]_v, \ldots, [\![t_n]\!]_v\big) \tag{3}$$

注记 1.1 (工程映射意义)*.* 定理 1.1 一针见血地指出了 Agent Runtime（宿主程序）的数学本质：

- LLM 的工作是在给定上下文的条件下，在项代数 $T_\Sigma(X)$ 上进行概率采样，生成一棵合法的语法树 $t \in T_\Sigma(X)$（在工程上序列化为 JSON 格式的 Tool Call）。
- Agent Runtime（如 Claude Code 的执行引擎或 MCP Server）就是同态映射 $[\![\cdot]\!]_v$！它负责解析语法树 t，并将其递归映射为操作系统载体集 $\mathcal{A}$ 上的真实算子调用 $\omega^{\mathcal{A}}$。

1.4 对偶视角：余代数（Coalgebra）与闭环随机控制

如果只有泛代数（从输入项归约到输出值：$\Sigma(A) \to A$），那么 Agent 只是一个单步编译器或计算器。为什么 Claude Code 能够连续执行数十步、根据报错信息自动修复代码？因为它的外层循环是一个与代数（Algebra）严格对偶的数学结构——**余代数（Coalgebra）**。

表 1: 泛代数（Algebra）与余代数（Coalgebra）在 Code Agent 中的对偶分工

维度	泛代数（Universal Algebra）	余代数（Coalgebra）
核心映射方向	$\alpha : F(A) \longrightarrow A$（构造 / 归约求值）	$\gamma : S \longrightarrow F(S)$（解构 / 状态展开）
核心数学概念	初始代数（Initial Algebra）、同态求值	终结余代数（Final Coalgebra）、互模拟（Bisimulation）
计算机科学对应	抽象语法树（AST）、表达式求值、函数调用	状态机、无限流（Streams）、反应式动力系统
Agent 内部角色	单步 Tool Call 的语法构造与环境执行	多轮 ReAct 观测-思考-行动闭环状态转移

在现代控制理论与动力系统视角下，设环境与上下文的联合状态空间为 $\mathcal{S}$，观测空间为 $\mathcal{O}$，项代数（控制动作空间）为 $\mathcal{U} = T_\Sigma(X)$。Code Agent 的完整运行轨迹由如下离散时间闭环反馈系统刻画：

$$\begin{cases} u_k \sim \pi_\theta(\cdot \mid h_k), & \text{（LLM 策略采样：从历史轨迹生成代数项 } u_k \in T_\Sigma(X)\text{ ）} \\ (s_{k+1}, o_{k+1}) = [\![u_k]\!]_{s_k}, & \text{（泛代数同态求值：在物理状态} s_k \text{ 上作用算子并产生新观测} o_{k+1}\text{）} \\ h_{k+1} = \Phi(h_k, u_k, o_{k+1}), & \text{（上下文状态转移：追加历史或执行 Compaction 降维投影）} \end{cases} \tag{4}$$

当目标判定谓词 $\mathrm{Goal}(s_k, o_k) = \mathrm{True}$（例如所有单元测试通过且编译零警告）时，系统达到不动点（Fixed Point）并停机输出。

2 大语言模型（LLM）原理与受限解码机制

理解了 LLM 在泛代数框架中的使命是 “在项代数 $T_\Sigma(X)$ 及自然语言序列空间上进行条件概率采样”，我们自然要问：LLM 是如何表示并学习这个复杂概率测度的？它又是如何保证采样的符号串一定符合泛代数签名 Σ 的语法规范的？

2.1 自回归生成的测度论与信息论原理

设有限词表（Vocabulary）为 $\mathcal{V}$（在现代模型中通常 $|\mathcal{V}| \in [32000, 150000]$）。记 $\mathcal{V}^*$ 为由 $\mathcal{V}$ 生成的自由幺半群（Free Monoid，即所有有限长度 Token 序列的集合）。

大语言模型的本质是在可测空间 $(\mathcal{V}^*, 2^{\mathcal{V}^*})$ 上定义一个参数化的概率测度 $\mathbb{P}_\theta$。对于任意序列 $\boldsymbol{w} = (w_1, w_2, \ldots, w_T) \in \mathcal{V}^T$，根据概率论中的乘法链式法则（Chain Rule of Probability），联合概率分布可以严格且无损地分解为条件概率的连乘积：

$$\mathbb{P}_\theta(w_1, w_2, \ldots, w_T) = \prod_{t=1}^{T} \mathbb{P}_\theta(w_t \mid w_1, w_2, \ldots, w_{t-1}) = \prod_{t=1}^{T} \mathbb{P}_\theta(w_t \mid \boldsymbol{w}_{<t}) \tag{5}$$

注意，这里没有任何“独立性假设”或“马尔可夫阶数截断假设”——它是严格恒等的！模型的全部难度被转化为：如何用神经网络参数 θ 构造一个映射 $f_\theta : \mathcal{V}^{t-1} \to \mathbb{R}^{|\mathcal{V}|}$，将任意历史前缀 $\boldsymbol{w}_{<t}$ 映射为下一个 Token 的未归一化对数几率（Logits）向量 $\boldsymbol{z}_t = f_\theta(\boldsymbol{w}_{<t})$，并通过玻尔兹曼（Softmax）映射转化为概率单纯形 $\Delta^{|\mathcal{V}|-1}$ 上的分布：

$$\mathbb{P}_\theta(w_t = v_i \mid \boldsymbol{w}_{<t}) = \frac{\exp(z_{t,i})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j})} \tag{6}$$

训练目标与 KL 散度最小化： 设真实人类语料与代码构成的经验分布为 $\mathbb{P}_{\text{data}}$。训练 LLM 的核心目标是最小化真实分布 $\mathbb{P}_{\text{data}}$ 与模型分布 $\mathbb{P}_\theta$ 之间的前向 Kullback-Leibler（KL）散度：

$$\begin{aligned}\min_\theta D_{\text{KL}}(\mathbb{P}_{\text{data}} \,\|\, \mathbb{P}_\theta) &= \min_\theta \mathbb{E}_{\boldsymbol{w}\sim\mathbb{P}_{\text{data}}}\left[\log \frac{\mathbb{P}_{\text{data}}(\boldsymbol{w})}{\mathbb{P}_\theta(\boldsymbol{w})}\right] \\ &= \min_\theta \underbrace{\mathbb{E}_{\boldsymbol{w}\sim\mathbb{P}_{\text{data}}}\left[-\sum_{t=1}^{T} \log \mathbb{P}_\theta(w_t \mid \boldsymbol{w}_{<t})\right]}_{\text{交叉熵损失（Cross-Entropy Loss）}\ \mathcal{L}_{\text{CE}}(\theta)} - H(\mathbb{P}_{\text{data}})\end{aligned} \tag{7}$$

由于真实数据分布的香农熵 $H(\mathbb{P}_{\text{data}})$ 与模型参数 θ 无关，因此**极大似然估计（Next-Token Prediction 的交叉熵损失最小化）在数学上严格等价于在函数空间中寻找逼近真实分布 $\mathbb{P}_{\text{data}}$ 的 KL 投影！**

2.2 词元化（Tokenization）：自由半群上的子字压缩

在输入神经网络之前，原始字节流必须经过词元化器（Tokenizer）映射为离散整数序列。现代 LLM 普遍采用字节级字节对编码（Byte-Level BPE）。

从代数视角看，初始词表 $\mathcal{V}_0 = \{0, 1, \ldots, 255\}$ 是单字节集合。BPE 算法在语料库上迭代执行以下步骤，直至词表大小达到预设上限 $|\mathcal{V}|$：

(1) 统计当前语料序列中相邻符号对 $(u, v) \in \mathcal{V}_k \times \mathcal{V}_k$ 的共现频数 $N(u, v)$。

(2) 选取频数最高的符号对 $(u^*, v^*) = \arg\max_{(u,v)} N(u, v)$，定义合并产生式 $w_{\text{new}} \leftarrow u^* \cdot v^*$（即自由半群上的乘法连接）。

(3) 更新词表 $\mathcal{V}_{k+1} = \mathcal{V}_k \cup \{w_{\text{new}}\}$，并将语料中所有的 (u^*, v^*) 替换为单一符号 w_{new}。

从信息论角度看，BPE 本质上是一种字典压缩算法：它将高频子串合并为单个 Token，从而降低序列长度 T，提升单个 Token 承载的平均信息量（香农熵逼近均匀分布），同时极大地节省后续 Transformer 的自注意力计算开销。

2.3 不可绕过的隐性基石：受限解码（Constrained Decoding）

这里有一个极为关键但常被普通教程忽略的问题：**既然 LLM 本质上是概率采样器，为什么当它调用 MCP 工具或输出结构化 JSON 时，不会因为哪怕一个毫秒的概率波动少写一个右花括号 } 或写错字段名，从而导致泛代数求值同态 $[\![\cdot]\!]$ 解析崩溃？**

答案是现代推理引擎（以及 Claude / OpenAI 的 Tool Use 底层）广泛采用了**基于形式文法与有限状态机（FSM / PDA）的受限解码（Constrained Decoding）**。

命题 2.1 (基于下推自动机掩码的语法保真采样)**.** *设泛代数签名 Σ 对应的 JSON Schema 被编译为一个上下文无关文法 $G = (V_N, \mathcal{V}, R, S_0)$ 或对应的确定性下推自动机（DPDA）状态转移函数 $\delta : Q \times \mathcal{V} \to Q \cup \{dead\}$。在解码第 t 步，设当前自动机处于语法状态 $q_{t-1} \in Q$。定义当前状态下的合法后继词元集（Valid Token Set）为：*

$$\mathcal{V}_{valid}(q_{t-1}) = \big\{ v \in \mathcal{V} \;\big|\; \delta(q_{t-1}, v) \neq dead \big\} \tag{8}$$

我们在 Softmax 归一化之前，构造掩码算子 $\mathcal{M}_{q_{t-1}} : \mathbb{R}^{|\mathcal{V}|} \to \mathbb{R}^{|\mathcal{V}|}$：

$$\tilde{z}_{t,i} = \begin{cases} z_{t,i}/\tau, & \text{若} v_i \in \mathcal{V}_{valid}(q_{t-1}) \\ -\infty, & \text{若} v_i \notin \mathcal{V}_{valid}(q_{t-1}) \end{cases} \tag{9}$$

随后按 $\tilde{\mathbb{P}}(w_t = v_i \mid \boldsymbol{w}_{<t}) = \frac{\exp(\tilde{z}_{t,i})}{\sum_j \exp(\tilde{z}_{t,j})}$ 进行采样。

通过将非法语法分支的 Logit 强制置为 $-\infty$（对应概率严格为 0），模型被强制约束在文法 G 生成的形式语言子流形 $L(G) \subset \mathcal{V}^*$ 上游走。这就是将“连续概率模型”与“离散泛代数项空间 $T_\Sigma(X)$”严丝合缝焊接在一起的底层工程秘诀！

3 Transformer 架构：希尔伯特空间上的算子动力学

3.1 Transformer 与 LLM 的精确关系

许多初学者常将 Transformer 与 LLM 混为一谈。用数学语言精准界定两者的关系：

- **LLM（大语言模型）是问题定义与统计建模框架**，即通过最大化自回归条件概率 $\prod_t \mathbb{P}(w_t \mid \boldsymbol{w}_{<t})$ 来建模序列分布（你完全可以用 RNN、LSTM 或状态空间模型 Mamba 来实现一个 LLM）。
- **Transformer** 是当前实现映射 $f_\theta : \mathcal{V}^{t-1} \to \mathbb{R}^{|\mathcal{V}|}$ 最成功的**参数化非线性算子族（Parameterized Operator Family）**。它彻底抛弃了时间步上的串行循环递归，改用内积空间上的非局部核注意力算子，从而实现了训练阶段的完全并行化与长程依赖的 $\mathcal{O}(1)$ 路径长度直接交互。

3.2 残差流（Residual Stream）：高维向量空间的总线动力学

设嵌入维度为 d_{model}（例如 $d_{\text{model}} = 4096$），序列长度为 T。输入序列首先通过词嵌入矩阵 $W_E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ 映射为初始状态矩阵 $X^{(0)} \in \mathbb{R}^{T \times d_{\text{model}}}$。

现代 Decoder-Only Transformer（如 Claude、LLaMA、Qwen 系列）由 L 层结构相同的算子块串联而成。理解 Transformer 最深刻的现代视角（源自 Anthropic 的机械可解释性研究）是**残差流（Residual Stream）视角**：

$$\begin{aligned} H^{(l)} &= X^{(l-1)} + \mathcal{A}^{(l)}\big(\text{RMSNorm}(X^{(l-1)})\big) \\ X^{(l)} &= H^{(l)} + \mathcal{F}^{(l)}\big(\text{RMSNorm}(H^{(l)})\big) \end{aligned} \qquad (l = 1, 2, \ldots, L) \tag{10}$$

其中 $\mathcal{A}^{(l)}$ 为多头自注意力算子（Multi-Head Self-Attention），$\mathcal{F}^{(l)}$ 为前馈神经网络算子（SwiGLU FFN），而 $\text{RMSNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}$ 是均方根归一化算子。

注记 3.1 (常微分方程欧拉离散化与读写总线). 观察上式，若记 $\Delta X^{(l)} = \mathcal{A}^{(l)}(\dots) + \mathcal{F}^{(l)}(\dots)$，则 $X^{(l)} = X^{(l-1)} + \Delta X^{(l)}$ 在数学形式上完全等价于常微分方程 $\frac{\mathrm{d}X(t)}{\mathrm{d}t} = G(X(t), t)$ 的前向欧拉（Forward Euler）离散化！这意味着：

(1) 主干道 $X^{(l)} \in \mathbb{R}^{T \times d_{\text{model}}}$ 是一条**共享通信总线（Shared Bus）**。

(2) 每一层的注意力头与 FFN 只是挂载在这条总线上的子模块：它们通过线性投影矩阵从总线中“读取”特定子空间的信息，经过非线性计算后，再通过输出投影矩阵以加法叠加的方式“写回”总线。

(3) 反向传播时，梯度流满足 $\frac{\partial \mathcal{L}}{\partial X^{(0)}} = \frac{\partial \mathcal{L}}{\partial X^{(L)}} \prod_{l=1}^{L} \left(I + \frac{\partial \Delta X^{(l)}}{\partial X^{(l-1)}} \right)$，单位算子 I 的存在构成了一条无损梯度高速公路，彻底消除了深层网络的梯度消失问题。

3.3 缩放点积注意力与方差稳定化证明

在残差流中，注意力算子 $\mathcal{A}^{(l)}$ 负责**跨位置（Token-to-Token）的信息路由与汇聚**，而 FFN 算子 $\mathcal{F}^{(l)}$ 负责**位置内部（Token-wise）的特征非线性变换与知识激活**。

对于单个注意力头，设输入矩阵为 $X \in \mathbb{R}^{T \times d_{\text{model}}}$，定义三个线性投影矩阵 $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 以及 $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$，生成查询（Query）、键（Key）和值（Value）矩阵：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{11}$$

因果缩放点积注意力（Causal Scaled Dot-Product Attention）定义为：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M_{\text{causal}} \right) V \tag{12}$$

其中因果掩码矩阵 $M_{\text{causal}} \in \mathbb{R}^{T \times T}$ 满足：当 $j \le i$ 时 $(M_{\text{causal}})_{ij} = 0$；当 $j > i$ 时 $(M_{\text{causal}})_{ij} = -\infty$，严防未来信息泄露。

定理 3.1 (缩放因子 $\frac{1}{\sqrt{d_k}}$ 的方差稳定化与防梯度饱和性质). *设查询向量 $\boldsymbol{q} \in \mathbb{R}^{d_k}$ 与键向量 $\boldsymbol{k} \in \mathbb{R}^{d_k}$ 的各个分量相互独立，且均服从均值为 0、方差为 1 的分布，即 $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$，$Var(q_i) = Var(k_i) = 1$。则未缩放的内积 $\langle \boldsymbol{q}, \boldsymbol{k} \rangle = \sum_{i=1}^{d_k} q_i k_i$ 满足：*

$$\mathbb{E}[\langle \boldsymbol{q}, \boldsymbol{k} \rangle] = 0, \quad Var(\langle \boldsymbol{q}, \boldsymbol{k} \rangle) = d_k \tag{13}$$

证明. 由独立性可知，$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$。对于方差：

$$\text{Var}\left(\sum_{i=1}^{d_k} q_i k_i \right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} \left(\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - (\mathbb{E}[q_i]\mathbb{E}[k_i])^2 \right) = \sum_{i=1}^{d_k} (1 \times 1 - 0) = d_k \tag{14}$$

若不除以 $\sqrt{d_k}$，当 d_k 较大（如 $d_k = 128$）时，内积的标准差高达 $\sqrt{128} \approx 11.31$。此时向量中的极值分量在经过指数函数 $\exp(\cdot)$ 放大后，会导致 Softmax 分布急剧退化为顶点处的 Dirac δ 分布（即某个分量接近 1，其余分量接近 0）。由于 Softmax 的雅可比矩阵为 $J_{\text{Softmax}}(\boldsymbol{p}) = \text{diag}(\boldsymbol{p}) - \boldsymbol{p}\boldsymbol{p}^\top$，当 $\boldsymbol{p} \to [1, 0, \dots, 0]^\top$ 时，$J_{\text{Softmax}}(\boldsymbol{p}) \to \mathbf{0}$，导致严重的梯度消失！除以 $\sqrt{d_k}$ 将内积方差严格归一化为 1，保证了梯度流的健康。 □

3.4 旋转位置编码（RoPE）：正交群上的相对位置内积不变性

纯粹的集合注意力运算具有置换等变性（Permutation Equivariance），无法感知词元的先后顺序。现代大模型普遍采用**旋转位置编码（Rotary Position Embedding, RoPE）**。

RoPE 的核心数学思想是：将维度为 d_k（设 d_k 为偶数）的实向量空间 $\mathbb{R}^{d_k}$ 同构映射到 $d_k/2$ 维复向量空间 $\mathbb{C}^{d_k/2}$。对于位置 $m \in \mathbb{N}$ 处的二维子向量 $\boldsymbol{q}^{(j)} = [q_{2j}, q_{2j+1}]^\top$（对应复数 $z_q^{(j)} = q_{2j} + \mathrm{i}q_{2j+1}$），我们在第 j 个二维平面上施加一个与位置 m 成正比的正交旋转变换 $R_{m,\theta_j} \in SO(2)$：

$$R_{m,\theta_j}\boldsymbol{q}^{(j)} = \begin{pmatrix} \cos(m\theta_j) & -\sin(m\theta_j) \\ \sin(m\theta_j) & \cos(m\theta_j) \end{pmatrix} \begin{pmatrix} q_{2j} \\ q_{2j+1} \end{pmatrix} \Longleftrightarrow z_q^{(j)} \mathrm{e}^{\mathrm{i}m\theta_j} \tag{15}$$

其中频率基底通常取 $\theta_j = B^{-2j/d_k}$（如 $B = 10000$ 或 500000）。

将位于位置 m 的查询向量与位于位置 n 的键向量做内积，利用正交旋转矩阵的转置性质 $R_{m,\theta_j}^\top R_{n,\theta_j} = R_{n-m,\theta_j}$，我们立刻得到：

$$\left\langle \boldsymbol{R}_m\boldsymbol{q}, \boldsymbol{R}_n\boldsymbol{k} \right\rangle = \sum_{j=0}^{d_k/2-1} \left(\boldsymbol{q}^{(j)}\right)^\top R_{m,\theta_j}^\top R_{n,\theta_j} \boldsymbol{k}^{(j)} = \mathrm{Re}\left(\sum_{j=0}^{d_k/2-1} z_q^{(j)} \left(z_k^{(j)}\right)^* \mathrm{e}^{\mathrm{i}(m-n)\theta_j} \right) \tag{16}$$

这个公式展示了 RoPE 的精妙之处：**虽然我们对位置 m 和位置 n 的向量分别进行了仅依赖自身绝对位置的独立编码（这对接下来要讲的 KV Cache 至关重要！），但它们的内积却严格只依赖于相对距离 $m-n$！**同时，多种频率 θ_j 的叠加相当于在位置空间上构造了一组多尺度傅里叶基，高频分量捕捉局部语法关联，低频分量捕捉长程跨文件语义关联。

3.5 SwiGLU 前馈网络：参数化键值关联记忆

在注意力层完成跨位置信息聚合后，每个位置独立通过 SwiGLU 前馈网络：

$$\mathcal{F}_{\mathrm{SwiGLU}}(\boldsymbol{x}) = \Big(\mathrm{Swish}_1(\boldsymbol{x}W_{\mathrm{gate}}) \odot (\boldsymbol{x}W_{\mathrm{up}}) \Big) W_{\mathrm{down}} \tag{17}$$

其中 $\mathrm{Swish}_\beta(z) = z \cdot \sigma(\beta z) = \frac{z}{1+\mathrm{e}^{-\beta z}}$，$\odot$ 表示逐元素哈达玛积（Hadamard Product）。从矩阵分解视角看，$W_{\mathrm{gate}}, W_{\mathrm{up}} \in \mathbb{R}^{d_{\mathrm{model}} \times d_{\mathrm{ff}}}$ 的每一列相当于一个“静态模式检测器（Key）”，当输入残差流向量 $\boldsymbol{x}$ 与其内积较大时，门控激活，随即将 $W_{\mathrm{down}} \in \mathbb{R}^{d_{\mathrm{ff}} \times d_{\mathrm{model}}}$ 中对应的行向量（“存储的事实或代码模板知识 Value”）线性组合写入残差流。因此，**预训练阶段学到的静态领域知识主要存储在 FFN 的权重矩阵中，而上下文动态推理能力主要由 Attention 层承载**。

3.6 核心工程基石：KV Cache 递推机制与硬件瓶颈分析

为什么 Claude Code 在处理长上下文时如此关注“Prompt Caching（前缀缓存）”？这直接源于自回归推理的 **KV Cache 矩阵分块递推机制**。

假设我们已经处理了长度为 $t-1$ 的序列，现在生成了第 t 个 Token，其输入行向量为 $\boldsymbol{x}_t \in \mathbb{R}^{1 \times d_{\mathrm{model}}}$。由于存在因果掩码 M_{causal}，前 $t-1$ 个位置的输出完全不依赖于第 t 个位置！因此，我们只需要计算第 t 个位置的注意力输出 $\boldsymbol{o}_t \in \mathbb{R}^{1 \times d_v}$。

将键矩阵和值矩阵按行分块：

$$K_{1:t} = \begin{pmatrix} K_{1:t-1} \\ \boldsymbol{k}_t \end{pmatrix} \in \mathbb{R}^{t \times d_k}, \quad V_{1:t} = \begin{pmatrix} V_{1:t-1} \\ \boldsymbol{v}_t \end{pmatrix} \in \mathbb{R}^{t \times d_v} \tag{18}$$

其中 $\boldsymbol{q}_t = \boldsymbol{x}_t W_Q$，$\boldsymbol{k}_t = \boldsymbol{x}_t W_K$，$\boldsymbol{v}_t = \boldsymbol{x}_t W_V$。第 t 步的注意力输出严格简化为向量-矩阵乘法（GEMV）：

$$\boldsymbol{o}_t = \mathrm{Softmax}\left(\frac{\boldsymbol{q}_t \left[K_{1:t-1}^\top,\ \boldsymbol{k}_t^\top\right]}{\sqrt{d_k}}\right)\begin{pmatrix} V_{1:t-1} \\ \boldsymbol{v}_t \end{pmatrix} \tag{19}$$

表 2: 大模型推理的两个阶段：Prefill（预填充）与 Decode（逐词解码）对比

特性维度	Prefill 阶段（处理输入 Prompt）	Decode 阶段（逐个生成输出 Token）
核心算子类型	矩阵-矩阵乘法（GEMM，$T\times d$ 乘 $d\times d$）	向量-矩阵乘法（GEMV，$1\times d$ 乘 $d\times t$）
单步计算复杂度	$\mathcal{O}(T^2 d_{\text{model}} + T d_{\text{model}}^2)$	$\mathcal{O}(t d_{\text{model}} + d_{\text{model}}^2)$（无 Cache 则为 $\mathcal{O}(t^2 d)$）
硬件瓶颈类型	**算力受限（Compute-Bound）**	**显存带宽受限（Memory-Bandwidth-Bound）**
算术强度 (FLOPs/Byte)	极高（权重加载一次，复用 T 个位置）	极低（每生成 1 个 Token 必须从显存搬运全部权重与 KV Cache）

显存占用精确公式： 以半精度（FP16/BF16，每个参数占 2 Bytes）为例，对于层数为 L、键值头数为 n_{kv}（采用分组查询注意力 GQA 时 $n_{\text{kv}} < n_{\text{head}}$）、头维度为 d_k、序列长度为 T、批次大小为 B 的模型，KV Cache 占用的显存字节数为：

$$\text{Mem}_{\text{KV}} = 2\ (\text{K 与 V}) \times 2\ (\text{Bytes}) \times B \times L \times n_{\text{kv}} \times d_k \times T = 4BLn_{\text{kv}}d_kT\ \text{Bytes} \tag{20}$$

这解释了为什么在设计 Agent 架构时，**保持 System Prompt 与工具声明前缀的绝对稳定（不随意改变顺序或插入动态时间戳）至关重要**——一旦前缀稳定，服务端即可直接复用已计算好的 $K_{1:T_{\text{prefix}}}$ 与 $V_{1:T_{\text{prefix}}}$（即 Prefix Caching），将原本极其昂贵的 Prefill 计算直接降级为内存指针命中，成本降低 90%、首字延迟（TTFT）降低 80% 以上！

4 MCP（Model Context Protocol）原理：泛代数签名的标准化总线

4.1 为什么需要 MCP？从 $M \times N$ 耦合灾难到统一接口代数

在 MCP 出现之前，假设世界上有 M 个大模型宿主应用（如 Claude Desktop、Cursor、Cline、各类 IDE 插件）和 N 个外部数据源或工具集（如 GitHub、PostgreSQL、本地文件系统、EDA 仪器控制接口）。每个宿主都要为每个工具编写私有的胶水代码与 JSON Schema 解析器，导致全行业面临 $\mathcal{O}(M \times N)$ 的集成复杂度爆炸。

Anthropic 提出的 **Model Context Protocol (MCP)** 本质上是 AI 时代的 **USB-C 硬件总线协议**（或类比编程语言领域的 LSP - Language Server Protocol）。它制定了一套标准化的通信规范，使复杂度骤降为 $\mathcal{O}(M + N)$。

从第 1 章的泛代数视角来看：**MCP 就是将外部环境 $\mathcal{A}$ 的多类代数签名 $\Sigma = (S, \Omega)$ 进行动态注册、发现与远程同态求值（Remote Homomorphic Evaluation）的工业级网络协议**。

4.2 Host-Client-Server 三层拓扑架构

MCP 严格定义了三个解耦的系统角色：

(1) **MCP Host（宿主应用）**：直接与用户交互并管理 LLM 上下文生命周期的主程序（例如 Claude Code 终端进程）。Host 内部维护着主控 Agent 循环和权限安全策略。

(2) **MCP Client（协议客户端）**：内嵌在 Host 内部的网络/进程通信组件。Host 会为每一个连接的 MCP Server 实例化一个独立的 Client，维持 $1:1$ 的有状态会话隔离。

(3) **MCP Server（能力服务端）**：独立的轻量级进程（可用 Python、TypeScript、Rust、C++ 编写）。它直接包裹底层操作系统资源、数据库或硬件驱动，向 Client 暴露标准化的泛代数接口。

4.3 三大核心原语的形式化代数定义

MCP 并非只有“工具调用”，它根据“谁发起控制”以及“是否产生环境副作用”将交互严格划分为三大原语（Primitives）：

表 3: MCP 三大核心原语的代数本质与控制归属对比

MCP 原语	控制发起方	泛代数与系统形式化定义	典型工程实例
Resources	应用程序 / 用户驱动	只读状态投影映射 $\pi_r : \texttt{EnvState} \to \texttt{Obs}$	数据库 Schema、日志文件、实时传感器波形
Prompts	用户显式触发	参数化自由项宏模板 $\tau(x_1, \ldots, x_k) \in T_\Sigma(X)$	`/review-pr`、标准化代码审计提示词模板
Tools	**模型自主决策驱动**	带副作用的代数算子 $\omega^{\mathcal{A}} : S \times A_1 \cdots \to S \times O$	执行 SQL 修改、发起 Git 提交、编译代码

4.4 传输层与 JSON-RPC 2.0 状态机交互全流程

MCP 的底层报文格式完全建立在无状态、轻量级的 **JSON-RPC 2.0** 协议之上，支持两种核心传输层（Transport Layer）：

- **Stdio Transport（标准输入输出管道）**：Host 作为父进程直接 `fork/exec` 启动本地 MCP Server 子进程。Client 向子进程的 `stdin` 写入一行 JSON-RPC 请求，从 `stdout` 读取响应（`stderr` 用于输出调试日志）。具备零网络开销、天然本地权限隔离的优势。
- **Streamable HTTP / SSE Transport**：面向远程微服务或云端部署，通过 HTTP POST 发送请求，并支持服务端通过 Server-Sent Events (SSE) 流式推送通知。

一个完整的 MCP 生命周期包含三个严格有序的阶段：

(1) **阶段一：能力协商握手（Initialization & Capability Negotiation）**
Client 发送 `initialize` 请求，声明自己支持的协议版本与客户端能力（如 `roots` 工作区边界、`sampling` 反向请求 LLM 能力）；Server 返回自身支持的原语集合（如声明支持 `tools` 和 `resources`）。随后 Client 发送 `notifications/initialized` 通知，握手完成。

(2) **阶段二：代数签名动态发现（Signature Discovery）**
Client 自动发送 `tools/list` 请求。Server 返回其所有算子的名称、自然语言描述以及极其严格的 `inputSchema`（遵循 JSON Schema 规范）。Host 收到后，将这些签名动态编译并注入到 LLM 的系统上下文前缀与受限解码器（FSM）中。

(3) **阶段三：算子调用与同态求值（Tool Invocation）**
当 LLM 在推理中采样出特定工具的调用项 $\omega(t_1, \ldots, t_n)$ 时，Host 拦截该输出，由对应的 MCP Client 封装为 `tools/call` 报文发送给 Server，并将返回的观测结果追加回残差流的上下文序列中。

以下是一段真实交互的 JSON-RPC 2.0 报文剖析：

Listing 1: MCP 协议中算子签名发现 (tools/list) 与同态求值 (tools/call) 的真实报文结构

```
// 1. Client -> Server: Request algebraic signature discovery
{"jsonrpc": "2.0", "id": 1, "method": "tools/list", "params": {}}

// 2. Server -> Client: Return many-sorted operator signatures
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "verify_pdf_build",
        "description": "Verify compiled PDF integrity and page count via poppler tools"
    ,
        "inputSchema": {
          "type": "object",
          "properties": {
            "pdf_path": {"type": "string", "description": "Path to compiled PDF file"},
            "min_pages": {"type": "integer", "description": "Minimum expected pages"}
          },
          "required": ["pdf_path", "min_pages"]
        }
      }
    ]
  }
}

// 3. Client -> Server: Evaluate ground term verify_pdf_build("main.pdf", 8)
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "verify_pdf_build",
```

```
    "arguments": {"pdf_path": "main.pdf", "min_pages": 8}
  }
}
```

5 Skill（智能体技能）原理：渐进式披露与派生算子抽象

既然我们已经有了强大的原子工具（如 Bash、文件读写）以及可扩展的 MCP 协议，为什么现代智能体（如 Claude Code）还要专门引入 **Skill（技能）** 这一架构层？它与 MCP 到底是什么关系？

5.1 为什么仅靠 MCP 会遭遇系统性瓶颈？

假设你为 Agent 挂载了 20 个 MCP Server，总计暴露了 150 个精细的工具算子，同时你还希望 Agent 掌握“如何编写符合本实验室规范的嵌入式驱动”、“如何排查 XeLaTeX 编译错误”、“如何自动部署个人学术网站”等数十套复杂的标准作业程序（SOP）。

如果将这 150 个工具的完整 JSON Schema 和数十套数千字的 SOP 全部硬编码塞进 System Prompt 中，会立刻引发两大灾难：

(1) **上下文窗口爆炸与算力浪费**：每次对话尚未开始，固定前缀就已经消耗了 $50,000+$ Tokens。

(2) **注意力稀释（Attention Dilution / Lost-in-the-Middle）**：回顾第 3 章的 Softmax 注意力公式 $\alpha_{ti} = \frac{\exp(q_t^\top k_i/\sqrt{d_k})}{\sum_{j=1}^{t}\exp(q_t^\top k_j/\sqrt{d_k})}$。当分母中的无关项 j 数量极其庞大时，每一个不相关的工具描述和 SOP 都会分走非零的注意力权重（即注意力噪声底座升高），导致模型对当前真正关键指令的信噪比（SNR）急剧下降，工具选错率呈指数上升！

5.2 Skill 与 MCP 的正交性：原子算子 vs. 派生宏算子

在泛代数框架下，MCP 与 Skill 的关系可以得到最清晰的数学界定：

- **MCP 定义的是基础代数签名 Σ 中的“原子算子（Primitive Operations）”**：它解决的是 **“能力边界问题（What atomic actions can the agent execute）”**。例如：提供一个连接示波器的接口、提供一个查询数据库的接口。
- **Skill 定义的是项代数 $T_\Sigma(X)$ 上的“派生算子 / 复合宏项（Derived Operations / Procedural Macros）”**：它解决的是 **“高阶编排与领域先验问题（How to compose atomic operations to solve a complex domain task）”**。

定义 5.1 (泛代数中的派生算子 / Derived Operation)**.** 设 $\Sigma = (S, \Omega)$ 为基础签名。一个 k 元派生算子 $\hat{\sigma}(x_1, \ldots, x_k)$ 是项代数 $T_\Sigma(\{x_1, \ldots, x_k\})$ 中的一个复合语法树（或带条件分支的策略子图）：

$$\hat{\sigma}(x_1, \ldots, x_k) = \omega_1\Big(\omega_2(x_1, c_1),\ \omega_3\big(\omega_4(x_2), \ldots, x_k\big)\Big) \tag{21}$$

在语义上，Skill 不必向操作系统添加任何新的二进制驱动，而是通过注入领域先验指令 $\mathcal{I}_{\text{skill}}$，强烈重塑 LLM 在项代数空间上的条件概率分布 $\mathbb{P}_\theta(u_{1:K} \mid h_t, \mathcal{I}_{\text{skill}})$，使其沿着经过人类专家验证的低熵高成功率轨迹演化！

5.3 Skill 的核心机制：三级渐进式披露（Progressive Disclosure）

为了彻底解决注意力稀释与上下文膨胀问题，Skill 架构采用了精妙的**三级渐进式披露（Progressive Disclosure，即上下文懒加载 Lazy Context Loading）**机制：

Level 1: 元数据路由层（Metadata Routing Index —常驻上下文）
在初始 System Prompt 中，系统**绝不加载**任何 Skill 的具体内容，而仅仅注入每个 Skill 的轻量级索引（仅包含 `name` 和一两句话的 `description` 触发条件，每个 Skill 仅消耗约 30 ∼ 50 Tokens）。这相当于操作系统内核中的“中断向量表”或“符号表”。

Level 2: 指令动态展开层（On-Demand Instruction Expansion —按需加载）
当用户的请求满足某个 Skill 的语义触发条件时，LLM 会自主调用一个元工具（如 `Skill` 加载器或直接读取 `.claude/skills/<skill-name>/SKILL.md`）。此时，长达数千字的详细 SOP、架构约束、边界条件与反例才会被动态推入当前活跃的残差流上下文窗口中。

Level 3: 附属资源与脚本执行层（Supplementary Resources & Scripts —按需挂载）
一个复杂的 Skill 目录除了核心的 `SKILL.md` 外，还可以包含子文档（如 `reference/api_spec.md`）、代码模板（`assets/template.tex`）以及确定性校验脚本（`scripts/verify.sh`）。`SKILL.md` 充当导航地图，指导 Agent 仅在需要时读取特定子文件或直接通过 Bash 执行校验脚本（脚本代码本身甚至无需进入上下文，只需将脚本的终端输出结果返回给模型！）。

Listing 2: 一个标准工业级 Skill 的目录结构与渐进式加载原理示意

```
.claude/skills/academic-paper-publisher/
|-- SKILL.md                    # Level 2: Core workflow instructions (loaded on
    trigger)
|-- reference/
|   |-- typography_rules.md     # Level 3: Detailed LaTeX formatting rules (loaded only
     if needed)
|-- templates/
|   |-- header_template.tex     # Level 3: Reusable boilerplate code
`-- scripts/
    `-- check_pdf_artifact.sh   # Level 3: Deterministic verification script (executed
    via Bash)
```

6 补全拼图：支撑现代 Code Agent 的三大隐性工程基石

掌握了泛代数、LLM、Transformer、MCP 和 Skill 之后，距离完全看透 Claude Code 这类顶级工业级 Agent 还差最后三块关键拼图。它们虽然很少出现在基础教科书中，但在实际运行中每时每刻都在发挥决定性作用。

6.1 隐性基石一：编译/测试反馈作为语法空间上的离散梯度（ReAct 闭环）

为什么大语言模型直接“一次性生成(Zero-shot)”大型工程代码往往漏洞百出，而放到 Claude Code 的 Agent 循环中却能写出完美运行的代码?

用最优化与控制论的视角审视：设目标正确代码为 $u^* \in T_\Sigma$，定义一个指示代价泛函（Cost Functional）$J : T_\Sigma \to \mathbb{R}_{\geq 0}$，例如编译错误数、未通过的测试样例数以及静态检查警告数的加权和。当 $J(u^*) = 0$ 时即达到全局最优解。

- **单次开环生成（Open-Loop Generation）**：相当于在没有任何反馈观测的情况下，仅凭先验分布 $\mathbb{P}_\theta(u \mid x_0)$ 盲目投掷飞镖，随着代码长度 L 增加，零错误概率呈指数衰减 $p^L \to 0$。
- **闭环 Agent 迭代（Closed-Loop Control）**：当 Agent 调用编译器（如 `xelatex` 或 `gcc`）或测试脚本时，编译器输出的报错信息（包含行号、错误类型、缺失符号）本质上是代价泛函 $J(u_k)$ 在离散语法空间上的**方向导数 / 局部梯度近似（Directional Subgradient Proxy）** $\nabla_{\text{syntax}} J(u_k)$!

LLM 接收该报错观测 o_k 后生成 `Edit` 补丁 Δu_k，在数学上严格对应于带反馈修正的迭代寻优过程：

$$u_{k+1} = \text{Patch}\big(u_k,\ \Delta u_k\big), \quad \text{其中} \Delta u_k \sim \pi_\theta\big(\cdot \ \big|\ u_k, \nabla_{\text{syntax}} J(u_k)\big) \tag{22}$$

这也解释了为什么**确定性的自动化检查脚本（而非模糊的视觉观察）**对于 Code Agent 如此重要：脚本提供的报错日志具有极高的信噪比和精确的局部坐标，能让模型在 $1 \sim 2$ 步迭代内精准收敛到 $J(u) = 0$ 的不动点。

6.2 隐性基石二：子智能体（Sub-agents）与函数调用栈帧隔离

在探索大型代码库或执行多阶段任务时，如果主 Agent 自己去执行 30 次 `grep` 和 `Read` 来寻找某个函数的定义，这 30 次中间探测产生的数万 Token 废料会永久堆积在主上下文历史 h_k 中，不仅污染后续推理的注意力分布，还会迅速耗尽 KV Cache。

现代 Code Agent 引入了 **Sub-agent（子智能体 / Task 工具）**机制。从计算机体系结构与泛代数角度看，启动一个 Sub-agent 完全等价于编程语言中的**函数调用栈帧（Call Stack Frame）与局部变量作用域隔离**：

$$o_{\text{distilled}} = \text{Proj}_{\text{return}}\Big(\text{RunCoalgebra}_{\text{sub}}\big(s_0^{\text{sub}} = \text{Prompt}_{\text{task}}\big)\Big) \tag{23}$$

子智能体在一个全新的、独立的上下文空间中展开数十步搜索与试错，待其内部余代数循环停机后，整個子上下文栈帧被直接销毁回收（Garbage Collected），仅将高度提炼的最终结论 $o_{\text{distilled}}$（通常仅数百 Tokens）作为单步算子的返回值交还给主 Agent!

6.3 隐性基石三：上下文压缩（Auto-Compaction）与充分统计量投影

当任务极其漫长、主上下文长度 T 逼近模型物理上限 $T_{\max}$ 时，系统会触发 **Compaction（自动摘要压缩）**算子 $\Pi_{\text{compact}} : \mathcal{V}^T \to \mathcal{V}^{T'}$（其中 $T' \ll T$）。

在数理统计与信息论中，$\Pi_{\text{compact}}(h_t)$ 的设计目标是寻找关于未来任务决策 $u_{t+1:\infty}$ 的**近似充分统计量（Approximate Sufficient Statistic）**，即最小化互信息损失：

$$\min_{\Pi_{\text{compact}}} \ I\big(h_t; u_{t+1:\infty}\big) - I\big(\Pi_{\text{compact}}(h_t); u_{t+1:\infty}\big) \quad \text{s.t.} \quad |\Pi_{\text{compact}}(h_t)| \leq T' \tag{24}$$

具体而言，Compaction 会丢弃冗长的原始文件读取回显、已修复的历史报错堆栈，但严格保留：(1) 用户的核心目标约束；（2）已修改的文件路径与关键架构决策；（3）当前尚未解决的待办事项 (TODO 状态)。

全篇核心知识体系脉络总结

至此，我们将现代 AI 智能体的全部核心概念串联成了一张严密的数学与工程网络：

- **Transformer** 利用残差流总线、$\frac{1}{\sqrt{d_k}}$ 缩放核注意力、RoPE 正交旋转编码与 SwiGLU 记忆网络，并在 **KV Cache** 的加持下，高效参数化了序列空间上的条件概率映射；
- **LLM** 通过最小化 KL 散度习得代码与自然语言的概率测度，并借助基于形式文法的 **受限解码（FSM Logit Masking）**，确保采样输出严格落在合法的语法流形内；
- **泛代数（Universal Algebra）** 将工具集定义为多类代数签名 Σ，将模型输出定义为自由项代数 $T_\Sigma(X)$，将环境执行定义为唯一求值同态 $[\![\cdot]\!]$；而 **余代数（Coalgebra）** 则刻画了多轮 **ReAct 闭环反馈控制**、**Sub-agent 栈帧隔离**与 **Compaction 降维投影**；
- 在工业协议与应用层，**MCP** 标准化了底层原子算子 Σ 的跨进程 JSON-RPC 2.0 发现与调用，而 **Skill** 则通过三级渐进式披露构造了高阶派生算子 $\hat{\sigma}$，在不稀释注意力的前提下赋予智能体专家级的领域工程能力。
